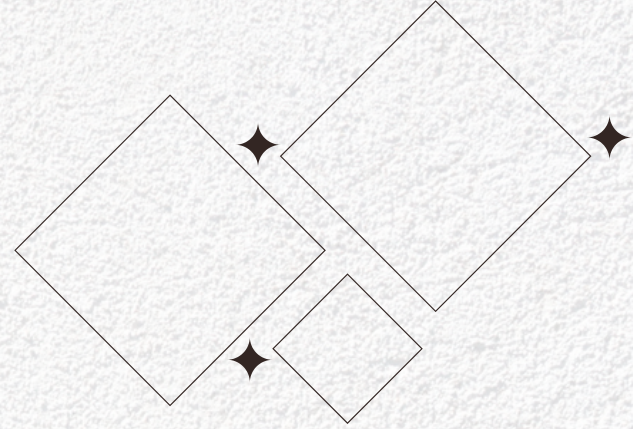


**UNIVERSIDAD
POLITÉCNICA**
DE SAN LUIS POTOSÍ

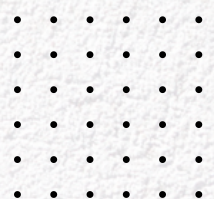
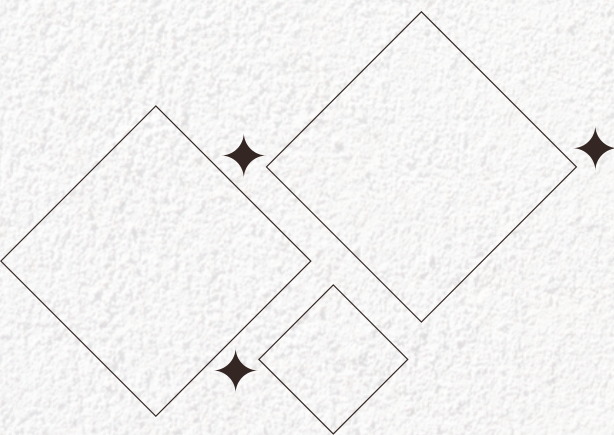


Laboratorio 7- SIMPLE CASE

CNO IV Programación WAP

Maestro: Servando López Contreras

Lopez Alcocer Monica Isabel - 179827



Índice

Índice	0
1. Ficha técnica del laboratorio.....	1
2. Objetivo del laboratorio.....	1
3. Marco teórico y fundamentación técnica	1
3.1 ¿Qué es el File Path Traversal?	1
3.2 Funcionamiento técnico.....	1
3.3 Variantes y técnicas de evasión de filtros	2
3.4 Impacto sobre la tríada CIA.....	2
3.5 Clasificación OWASP / CWE	2
3.6 Principios de defensa	2
4. Explicación técnica de la vulnerabilidad (caso específico).....	2
5. Procedimiento paso a paso.....	3
6. Evidencias — Capturas de pantalla obligatorias	5
6.1 Reconocimiento inicial (Site map en Burp Suite)	5
6.2 Request interceptado.....	5
6.3 Payload utilizado	6
6.4 Confirmación «Lab Solved»	6
7. Explicación del payload	6
8. Mitigación recomendada	7
9. Conclusión	7

1. Ficha técnica del laboratorio

Nombre	File path traversal, simple case
Plataforma	PortSwigger — Web Security Academy
URL	https://portswigger.net/web-security/file-path-traversal/lab-simple
Nivel	Apprentice
Tipo de explotación	File Path Traversal (Directory Traversal / LFI de lectura)
Clasificación	CWE-22 · OWASP Top 10 2021 → A01: Broken Access Control
Herramienta	Burp Suite Community Edition (Proxy + Repeater)
Objetivo de resolución	Leer el archivo <code>/etc/passwd</code> del servidor

2. Objetivo del laboratorio

La tienda en línea del laboratorio carga la imagen de cada producto mediante una función que recibe el **nombre del archivo** como parámetro. El objetivo es aprovechar que esa función construye la ruta del archivo directamente a partir de la entrada del usuario, sin validarla, para **salir del directorio de imágenes y leer un archivo arbitrario del sistema operativo**: concretamente `/etc/passwd`. Recuperar y mostrar su contenido es la condición que la plataforma evalúa para marcar el laboratorio como «*Solved*».

3. Marco teórico y fundamentación técnica

3.1 ¿Qué es el File Path Traversal?

El **File Path Traversal** —también llamado *Directory Traversal* o «`../` attack»— es una vulnerabilidad de control de acceso que permite a un atacante leer (y en algunos casos escribir) archivos ubicados fuera del directorio previsto por la aplicación web. Ocurre cuando el servidor utiliza entrada controlada por el usuario para construir una ruta del sistema de archivos **sin canonicalizarla ni validarla**, de modo que secuencias de recorrido como `../` permiten «subir» niveles en el árbol de directorios hasta alcanzar la raíz y, desde ahí, descender a cualquier ruta del sistema.

3.2 Funcionamiento técnico

Las aplicaciones suelen servir recursos con código equivalente a:

```
ruta = "/var/www/images/" + parametroUsuario; // p. ej. parametroUsuario = "25.jpg"
leer(ruta); // -> /var/www/images/25.jpg
```

En un sistema tipo UNIX/Linux la secuencia `..` representa el «directorio padre». Si el atacante envía `../../../../etc/passwd`, el sistema operativo resuelve la ruta así:

```
/var/www/images/ + ../../../../etc/passwd
/var/www/images/.. -> /var/www/
/var/www/..       -> /var/
/var/..           -> /
Resultado final:  -> /etc/passwd
```

Como el sistema de archivos no permite subir por encima de la raíz /, incluir «de más» secuencias ../ es inofensivo: por eso el atacante añade varias para garantizar que, sin importar la profundidad real del directorio base, siempre se alcance la raíz.

3.3 Variantes y técnicas de evasión de filtros

Este laboratorio es el «caso simple» (sin defensas). Sin embargo, en escenarios reales suelen existir controles parciales, y la familia de la vulnerabilidad incluye variantes que los evaden:

Defensa del servidor	Técnica de evasión típica
Ninguna (caso de este lab)	../../etc/passwd
Bloquea rutas relativas	Ruta absoluta: /etc/passwd
Elimina ../ una sola vez (no recursivo)	Anidado://....//....//etc/passwd
Filtra ../ por cadena literal	Codificación URL: ..%2f o doble: ../252f
Valida que empiece por una carpeta esperada	Prefijo + recorrido: /var/www/images/../../etc/passwd
Valida que termine en extensión (.png)	Byte nulo (sistemas legacy): ../../etc/passwd%00.png

3.4 Impacto sobre la tríada CIA

- **Confidencialidad (impacto alto):** permite leer archivos sensibles del servidor —credenciales, código fuente, archivos de configuración con contraseñas de bases de datos, claves privadas SSH, tokens— lo que puede derivar en compromisos mayores.
- **Integridad (impacto condicional):** si la funcionalidad vulnerable permite escritura (subida/guardado de archivos), el recorrido puede usarse para sobrescribir archivos y llegar a ejecución de código.
- **Disponibilidad (impacto indirecto):** la información obtenida (credenciales, rutas internas) suele ser el trampolín para ataques que sí afectan la disponibilidad del servicio.

3.5 Clasificación OWASP / CWE

- **CWE-22:** Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal').
- **OWASP Top 10 2021 — A01: Broken Access Control**, categoría que agrupa el acceso a recursos fuera de los límites autorizados.

3.6 Principios de defensa

El diseño seguro se apoya en tres ideas: no confiar en la entrada del usuario para construir rutas, validar contra una lista blanca conocida, y canonicalizar la ruta final para comprobar que permanece dentro del directorio base permitido. Se detallan en la sección 8.

4. Explicación técnica de la vulnerabilidad (caso específico)

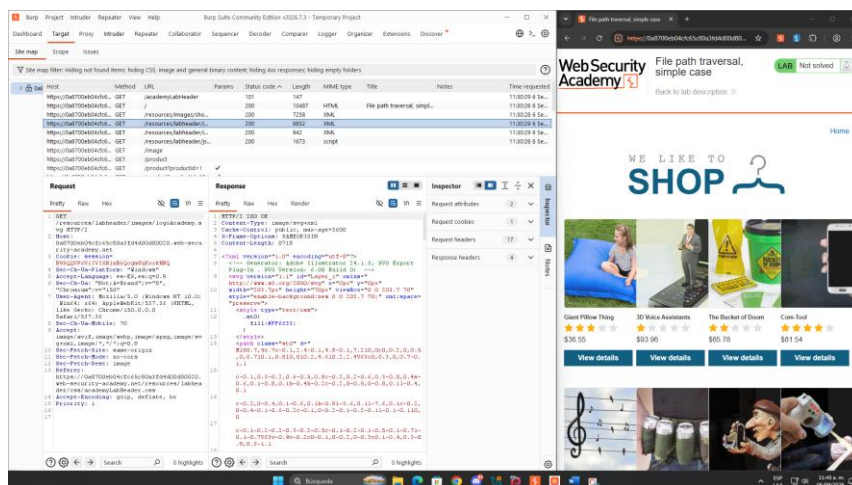
En este laboratorio, al abrir el detalle de un producto («View details»), el navegador solicita la imagen mediante un endpoint dedicado observable en el *Site map* de Burp: /image. La petición real tiene la forma:

```
GET /image?filename=25.jpg HTTP/2
Host: LAB-ID.web-security-academy.net
```

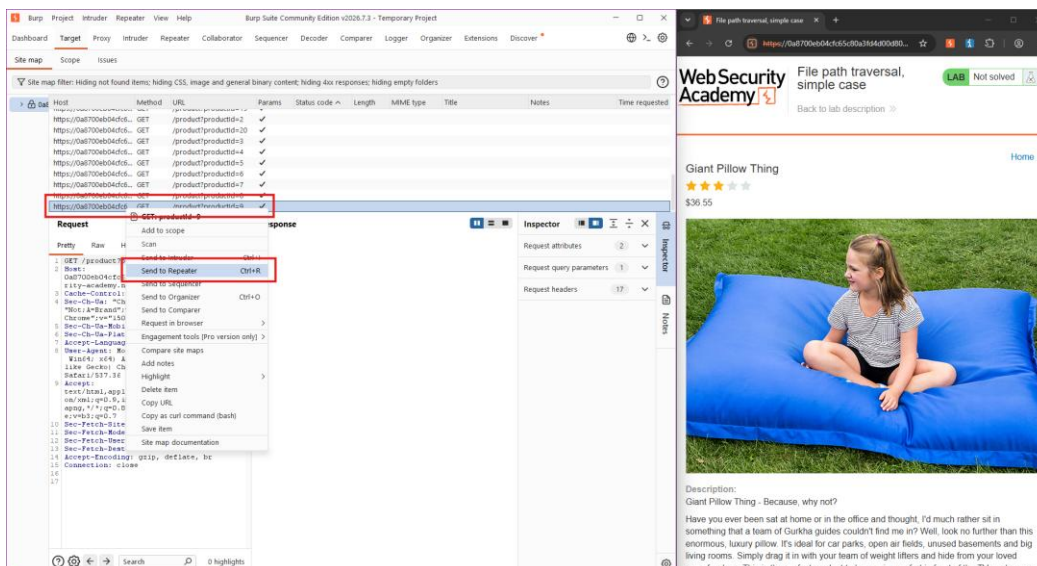
La aplicación toma el valor del parámetro filename y lo concatena directamente a la ruta del directorio de imágenes para leer el archivo del disco. **No aplica ninguna validación, canonicalización ni lista blanca** sobre ese valor. Por lo tanto, cualquier secuencia de recorrido introducida en filename es interpretada literalmente por el sistema de archivos, lo que permite abandonar el directorio de imágenes y leer archivos arbitrarios del servidor.

5. Procedimiento paso a paso

1. **Iniciar el laboratorio** desde el botón «Access the lab» y abrir la tienda en el navegador integrado de Burp (Proxy → Intercept → «Open Browser»), para que todo el tráfico pase por el proxy.



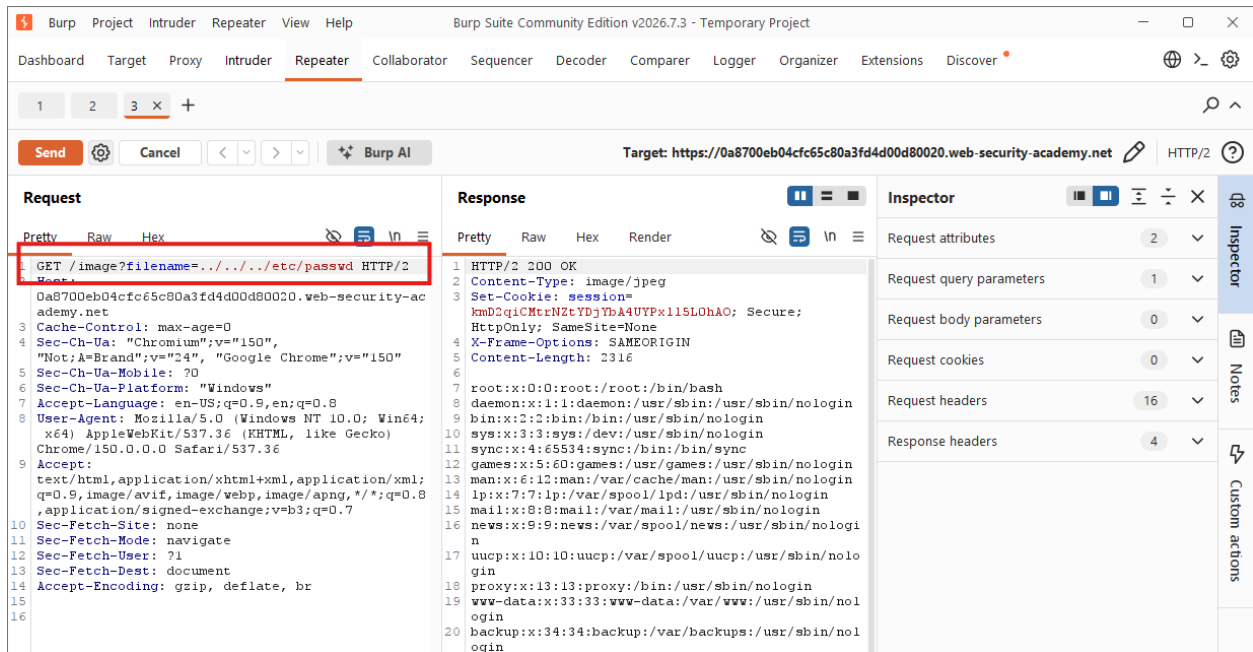
2. **Reconocimiento.** Navegar por la tienda y abrir un producto con «View details». En Burp, revisar Target → Site map y localizar el endpoint /productId junto con las peticiones de producto
3. **Localizar la petición de la imagen.** En el Site map o en Proxy → HTTP history, seleccionar la petición GET /product?productId=9.jpg.
4. **Enviar a Repeater.** Clic derecho sobre la petición → «Send to Repeater» (o Ctrl+R). Repeater permite modificar y reenviar la petición sin depender de la intercepción en vivo.



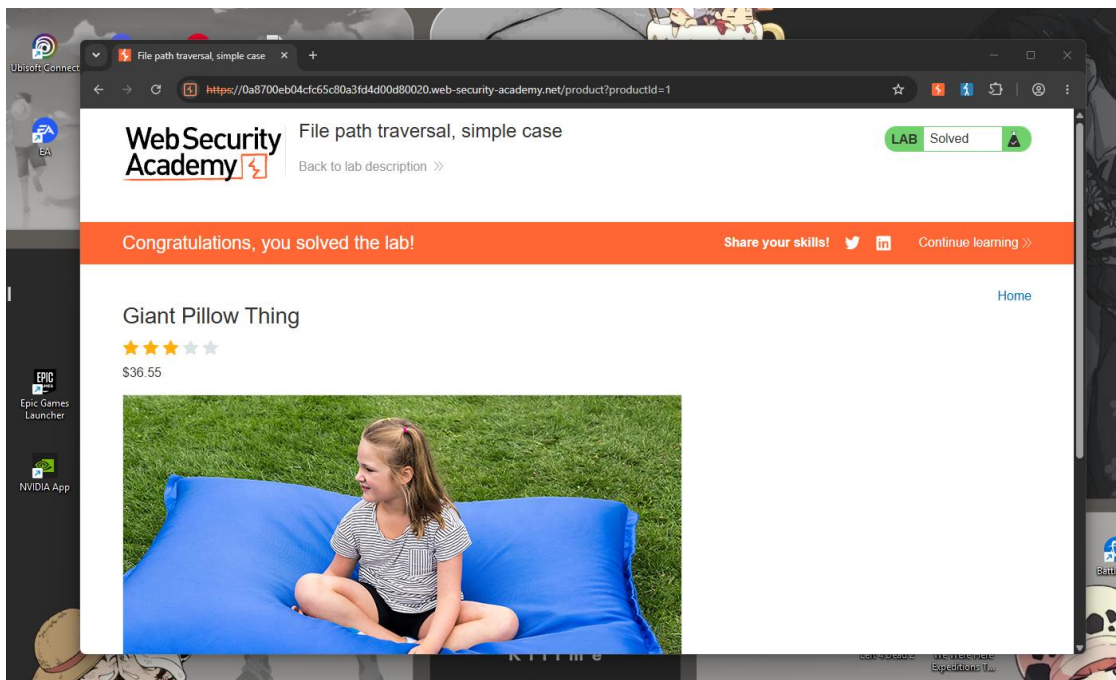
5. **Interceptar / modificar el parámetro.** En Repeater, sustituir el valor del parámetro filename por el payload de recorrido:

GET /product?productId=9 HTTP/2

6. **Enviar la petición con «Send»** y observar la respuesta en el panel derecho.



7. **Confirmar la explotación.** La respuesta devuelve un código 200 OK y, en el cuerpo, el contenido de `/etc/passwd` (líneas del tipo `root:x:0:0:root:/root:/bin/bash`). Esto demuestra la lectura de un archivo del sistema fuera del directorio de imágenes.
8. **Verificar «Lab Solved».** Al recuperar el archivo, la plataforma cambia el estado del laboratorio de «Not solved» a «Solved» (banner verde en la parte superior).



6. Evidencias — Capturas de pantalla

A continuación, se incluye la captura de reconocimiento realizada durante la práctica.

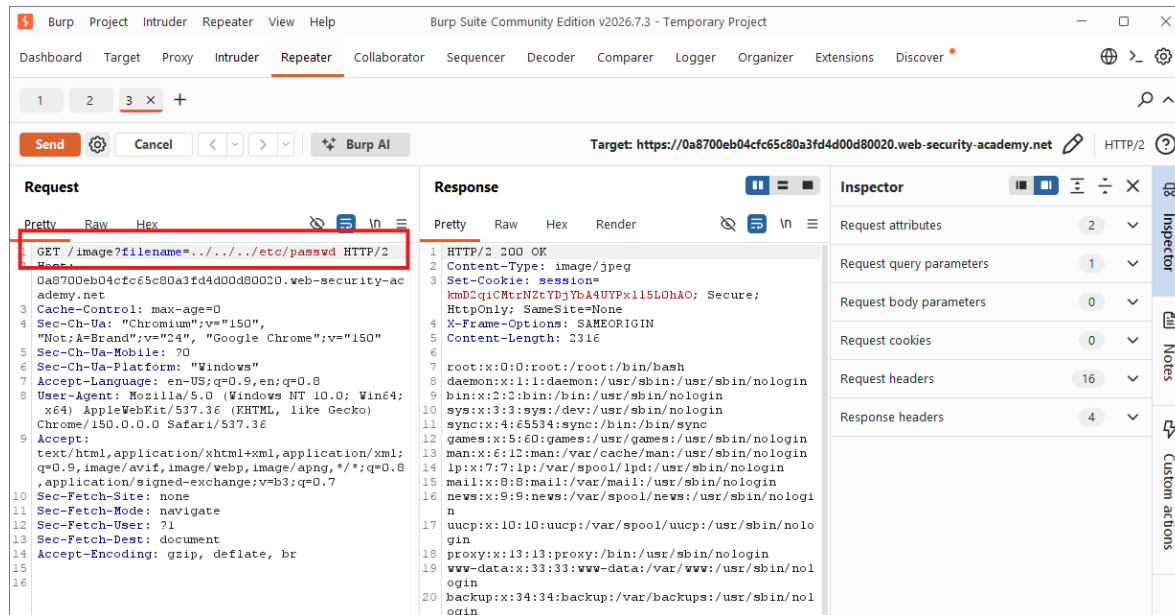
6.1 Reconocimiento inicial (Site map en Burp Suite)

The screenshot displays two windows side-by-side. On the left is the Burp Suite interface, showing the 'Site map' tab. The site map lists various URLs and their corresponding HTTP methods and status codes. The 'Request' tab is also visible, showing a detailed view of an HTTP request. On the right is a web browser window showing the 'File path traversal, simple case' lab page from WebSecurity Academy. The page features a 'SHOP' section with various products like 'Giant Pillow Thing', '3D Voice Assistants', 'The Bucket of Doom', and 'Com-Tool'.

6.2 Request interceptado

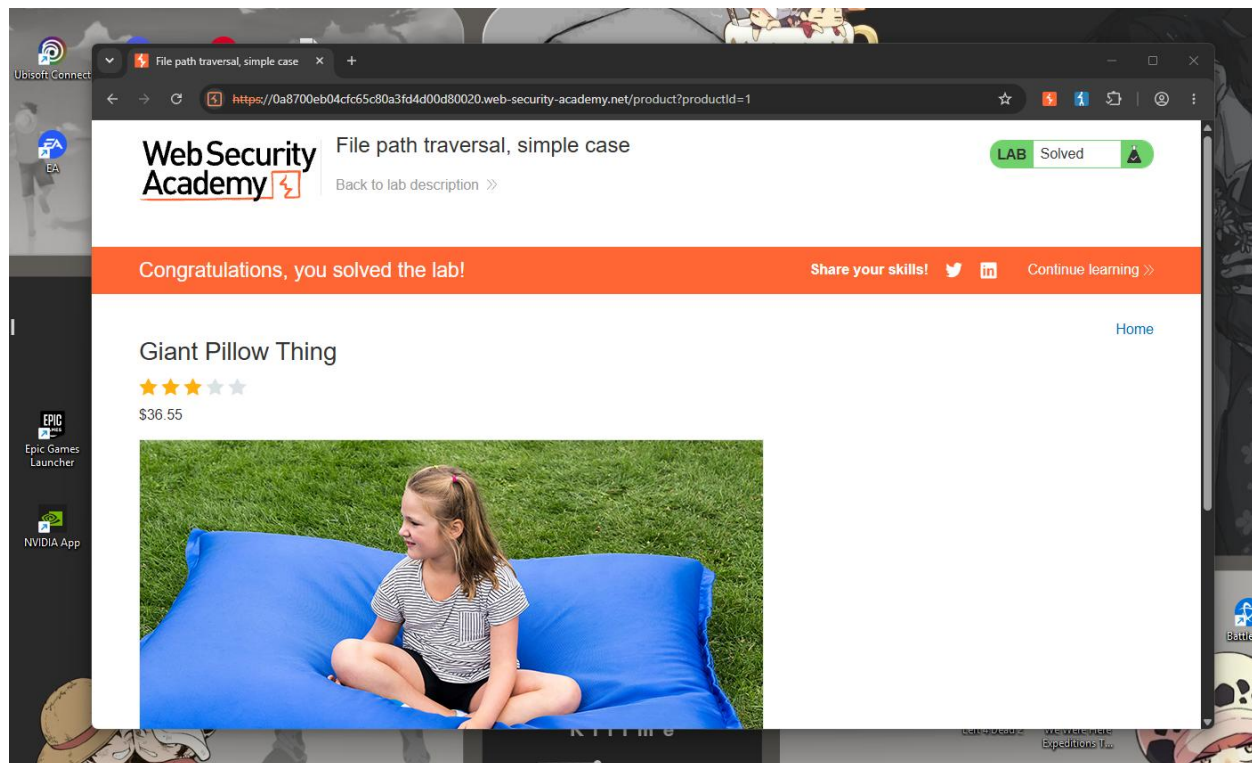
The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. A request is intercepted and displayed in the 'Request' tab. The request is a GET request to the URL 'https://0a8700eb04cfc65c80a3fd4d00d80020.web-security-academy.net/product?productId=9'. The 'Response' tab is also visible, showing the response from the server. The 'Inspector' tab is open, showing the request headers and body parameters.

6.3 Payload utilizado



The screenshot shows the Burp Suite interface with the Repeater tab selected. The target is `https://0a8700eb04cfc65c80a3fd4d00d80020.web-security-academy.net`. The request is a GET to `/image?filename=../../../../etc/passwd`. The response is a 200 OK with a Content-Type of `image/jpeg`. The Inspector panel on the right shows the request and response details.

6.4 Confirmación «Lab Solved»



The screenshot shows the Web Security Academy website. The page title is "File path traversal, simple case". The status is "LAB Solved". The page displays a congratulatory message: "Congratulations, you solved the lab!". Below the message, there is a product listing for "Giant Pillow Thing" with a price of \$36.55. The product image shows a young girl sitting on a large blue pillow outdoors.

7. Explicación del payload

`../../../../etc/passwd`

Componente	Función dentro del payload
../	Cada aparición sube un nivel en el árbol de directorios (al directorio padre).
../../../../	Tres saltos hacia arriba: suficientes para alcanzar la raíz / desde el directorio de imágenes. Añadir de más es inofensivo porque no se puede subir por encima de la raíz.
/etc/passwd	Ruta absoluta, ya desde la raíz, del archivo objetivo. En Linux es un archivo estándar y legible por todos, ideal como prueba de concepto.

Funciona porque el servidor concatena el parámetro a la ruta base (/var/www/images/ + filename) y entrega esa ruta al sistema de archivos sin canonicalizarla. El sistema operativo resuelve las secuencias ../ de forma nativa, de modo que la ruta efectiva pasa a ser /etc/passwd. No se requiere ninguna técnica de evasión porque, en este «caso simple», no existe filtro alguno que evadir.

¿Por qué /etc/passwd? Es un archivo presente en cualquier sistema Linux, con permisos de lectura para todos los usuarios y un formato reconocible. Por eso se emplea como prueba de concepto estándar: si se lee, queda demostrada la lectura arbitraria de archivos, sin causar daño ni exponer secretos críticos.

8. Mitigación recomendada

La defensa más robusta es evitar por completo pasar entrada del usuario a las APIs del sistema de archivos. Cuando no sea posible, se recomienda aplicar defensa en profundidad combinando varios controles:

- **Referencias indirectas:** no exponer nombres de archivo reales. Asignar un identificador (p. ej. `imageId=25`) que el servidor mapee internamente a la ruta física mediante una tabla o lista blanca.
- **Validación por lista blanca:** aceptar únicamente valores esperados (por ejemplo, nombres que cumplan una expresión regular estricta como `^[a-zA-Z0-9_-]+\.(png|jpg)$`) y rechazar cualquier otro.
- **Canonicalización y verificación de contenedor:** resolver la ruta a su forma canónica (p. ej. `realpath / File.getCanonicalPath`) y verificar que el resultado siga estando dentro del directorio base permitido antes de abrir el archivo.
- **Neutralizar secuencias de recorrido:** decodificar completamente la entrada y rechazar (no solo «limpiar») cualquier `../`, `..\` o sus variantes codificadas.
- **Principio de mínimo privilegio:** ejecutar el servicio web con un usuario sin permisos de lectura sobre archivos sensibles del sistema, limitando el impacto aun si el recorrido tiene éxito.

Nota: filtrar por lista negra (bloquear `../`) es insuficiente por sí solo, ya que puede evadirse con codificación URL, doble codificación o secuencias anidadas (ver sección 3.3). La validación debe ser por lista blanca y complementarse con la verificación del directorio contenedor.

9. Conclusión

El laboratorio demuestra, en su forma más pura, cómo la falta de validación de la entrada al construir rutas del sistema de archivos deriva en lectura arbitraria de archivos. Con un único parámetro manipulado —`filename=../../../../etc/passwd`— y sin ninguna técnica de evasión, se logró leer `/etc/passwd`, confirmando la vulnerabilidad y resolviendo el laboratorio. El caso evidencia por qué el control de acceso a recursos (OWASP A01) debe apoyarse en referencias indirectas, listas blancas y canonicalización, y por qué nunca debe confiarse en la entrada del usuario para acceder al sistema de archivos.